Here is my assignment instruction:
Assignment: Using a New York Times API in R

The New York Times provides multiple public APIs via its developer portal (NYT Developer Network). To use any NYT API, you must first create an account and obtain an API key.

Your task:

Choose one New York Times API endpoint (e.g., Article Search, Most Popular, Books, etc.). You should use your data analysis to ask and attempt to answer an interesting question of your choosing. Examples: "What are the top five best selling hard cover books?", "Which newspaper sections have produced the most popular articles, and how has this changed compared to X years ago?", etc.
In R, write code to:
Authenticate using your API key. Never hard code an API key. There are a number of good ways to do this. For example: Store it in an environment variable (e.g., Sys.getenv("NYT_API_KEY")).
Make a request to the endpoint
Parse the JSON response
Transform the result into a clean R data frame (tibble is fine)
Deliverable

A single Quarto (.qmd) document that includes:
The API you selected (endpoint + brief description)
The request you made (parameters used)
Code that returns a tidy data frame
A short note describing any data-cleaning decisions (e.g., nested fields, missing values)
NYT API page screen shot

Create question for me and give me a suitable API

Here are a few **strong, assignment-ready ideas** with matching NYT APIs. I'll recommend one as the *best fit* based on simplicity + good analysis potential in R.

✅ Best Choice (Recommended)

📊 Question:

"Which New York Times sections produce the most popular articles, and what types of content dominate the top results?"

🔌 API:

Most Popular API (Most Viewed Articles)

- Endpoint: `/svc/mostpopular/v2/viewed/{period}.json`
- Period options: `1`, `7`, or `30` days

👍 Why this is ideal:

- Clean JSON (easy parsing in R)
- No complex nesting
- Perfect for grouping + visualization
- Works well with `dplyr`, `ggplot2`

🔍 What you can analyze:

- Most frequent **sections** (e.g., World, Tech, Politics)
- Top **keywords/topics**
- Average views or engagement proxies
- Trends across 7 vs 30 days (optional)

🟡 Alternative Option 1 (Slightly More Advanced)

📊 Question:

"What topics and keywords appear most frequently in NYT articles over time?"

🔌 API:

Article Search API

- Endpoint: `/svc/search/v2/articlesearch.json`

👍 Pros:

- Very rich dataset (keywords, dates, authors)

⚠️ Cons:

- Nested JSON → requires more cleaning (`purrr`, `unnest`)
- Pagination handling needed

🟢 Alternative Option 2 (Very Easy & Clean)

📊 Question:

"What are the current top 10 hardcover fiction books, and which publishers dominate the list?"

🔌 API:

Books API (Best Sellers)

- Endpoint: `/svc/books/v3/lists/current/hardcover-fiction.json`

👍 Pros:

- Extremely clean JSON
- Minimal wrangling
- Good for quick completion

⚠️ Cons:

- Limited depth for analysis

💡 My Recommendation for You

Given your background in **data analysis + R + tidyverse**, go with:

👉 Most Popular API + Section Analysis question

It hits the sweet spot:

- Not too simple
- Not too messy

- Good storytelling potential in your report

📦 Example Request (you'll use this in R)

```r
url <- paste0(
```

‹/› R

📊 Expected Data Fields

You'll get:

- `title`
- `section`
- `published_date`
- `byline`
- `url`

✨ Bonus Ideas (if you want to impress)

- Compare **7-day vs 30-day popularity**
- Extract and analyze **media metadata**
- Create a **bar chart of top sections**

If you want, I can next:
✅ Write the full **Quarto (.qmd)** for you
✅ Provide **complete R code (API → tidy dataframe → visualization)**
✅ Help you generate a **clean narrative + conclusion**

Just tell me 👍

Give me a topic for the following: You should use your data analysis to ask and attempt to answer an interesting question of your choosing

The API should be MOst popular API but avoid the one example that professor gave

Got it—you want something **original**, still using the *Most Popular API*, but not the obvious "which sections are most popular."

Here are some **strong, unique topic ideas** you can pick from 👇

✅ Best Topic (Recommended)

📊 Question:

“Do articles with images receive more engagement than those without in the New York Times Most Popular list?”

💡 Why this stands out:

- Not a common/obvious question
- Uses a **less obvious field** (`media`)
- Shows deeper understanding of JSON structure
- Easy to analyze in R

🔍 What you’ll do:

- Extract `media` field
- Create a binary variable:
  - `Has_Image = Yes/No`
- Compare:
  - Count of articles
  - (Optional) Differences across sections

🟡 Alternative Topic 1

📊 Question:

“How does article popularity vary by day of the week?”

💡 Insight:

- Are weekends vs weekdays different?
- Uses `published_date`

🔍 Analysis:

- Convert date → weekday
- Count frequency by weekday

🟢 Alternative Topic 2

📊 Question:

"Do articles written by staff writers vs agencies differ in popularity?"

💡 Insight:

- Use `byline` field
- Compare patterns like:
  - "By [Name]" vs "Reuters", "AP News"

🔵 Alternative Topic 3

📊 Question:

"What is the average title length of popular NYT articles, and does it vary by section?"

💡 Insight:

- Creative + text analysis angle

🔍 Analysis:

- Compute `nchar(title)`
- Compare across sections

🔵 Alternative Topic 4

📊 Question:

"Which keywords/topics appear most frequently in the most popular NYT articles?"

💡 Insight:

- Uses `adx_keywords`
- Great for simple text frequency analysis

🚀 My Suggestion for You

Go with:

👉 **"Do articles with images receive more engagement than those without?"**

Why?

- Unique (professor likely won't see duplicates)
- Clean logic
- Shows **data wrangling skills (nested JSON)**
- Easy to explain in your report

If you want next step, I can:
✅ Write your **R code (API → clean dataframe → analysis)**
✅ Help you phrase **introduction + conclusion**
✅ Generate a full **Quarto (.qmd) template** ready to submit

For topic - "Do articles with images receive more engagement than those without in the New York Times Most Popular list?"

give me step-by-step R instruction

Great choice—that's a **smart and non-obvious question**, and it will let you show real data wrangling skills 👌

Below is a **clear step-by-step R workflow** you can directly use in your Quarto file.

🧩 Step 1: Install & Load Required Packages

```r
install.packages(c("httr", "jsonlite", "dplyr", "tidyr", "ggplot2"))

library(httr)
library(jsonlite)
```

‹/› R

```r
library(tidyr)
library(ggplot2)
```

🔐 Step 2: Store and Access API Key (Important!)

⚠️ Never hardcode your API key.

Option A (Recommended – Environment Variable)

‹/› R

Then retrieve:

‹/› R

🌐 Step 3: Make API Request

```r
url <- paste0(
  "https://api.nytimes.com/svc/mostpopular/v2/viewed/7.json?api-key=",
  api_key
)
```

‹/› R

```
data_json <- content(response, as = "text", encoding = "UTF-8")
data <- fromJSON(data_json, flatten = TRUE)
```

📦 Step 4: Extract Relevant Data

```r
articles <- data$results

df <- articles %>%
  select(
    title,
    section,
    media
  )
```

🧹 Step 5: Create "Has Image" Variable

The `media` field is a **list column**, so we check if it contains data.

```r
df <- df %>%
  mutate(
```

🧼 Step 6: Clean Data

```r
df_clean <- df %>%
  select(-media) %>%
)
```

R

📊 Step 7: Analyze Engagement (Proxy)

Since actual "views" aren't provided, we treat **appearance in Most Popular list** as engagement.

Count articles with vs without images:

```r
image_summary <- df_clean %>%
```

R

📊 Step 8: Compare Across Sections (Optional but Strong)

```r
section_analysis <- df_clean %>%
```

R

```
summarise(count = n(), .groups = "drop")
```

📈 Step 9: Visualization

Simple comparison plot:

```R
ggplot(image_summary, aes(x = has_image, y = count)) +
  geom_bar(stat = "identity") +
  labs(
    title = "Do Articles with Images Appear More in NYT Most Popular?",
  )
```

Section-wise comparison:

```R
ggplot(section_analysis, aes(x = section, y = count, fill = has_image)) +
  geom_bar(stat = "identity", position = "dodge") +
  coord_flip() +
  labs(
    title = "Image Presence by Section"
  )
```

📝 Step 10: Interpretation (Write in Your Report)

You can say something like:

- Articles with images appear **more frequently** in the most popular list (if true)
- Suggests **visual content may contribute to engagement**
- However, note:
  - This is **observational**, not causal
  - NYT may already prioritize image-rich content

📓 Step 11: Cleaning Notes (Required for Assignment)

Include something like:

- The `media` field was nested → converted into a binary variable (`has_image`)
- Missing media entries treated as “No image”
- Dates converted to proper `Date` format

✅ You’re Done!

If you want, I can now:
✅ Turn this into a **complete Quarto (.qmd) file**
✅ Add **nice narrative (intro, methods, conclusion)**
✅ Help you include a **screenshot section for NYT API page**

Just tell me 👍

Create an approach submission for this assignment in Quarto document. Following is the template which you should exactly do. Also, add what will be the approach for this data analysis:
Objective

The objective of this assignment is to create HTML and JSON data files containing information about selected adventure books and import them into R for analysis.

Approach

For this assignment, I created a small dataset containing three adventure books written by authors whose works I enjoy reading, including Alexandre Dumas, James Rollins, and H. Rider Haggard.

The selected books are:

- **The Three Musketeers**

- **Sandstorm**
- **King Solomon's Mines**

The dataset contains the following variables:

- **Title** – Name of the book
- **Authors** – Author or authors of the book
- **Publication Year** – Year the book was originally published
- **Publisher** – Publishing company responsible for the book
- **Genre** – Literary genre of the book

The same dataset is stored in two different formats:

1. **HTML table** – representing structured tabular data often found on websites.
2. **JSON file** – representing hierarchical data commonly used by APIs

Working with both formats will demonstrate how structured information from web sources can be imported and converted into R data frames.

Anticipated Challenges

One of the main challenges when working with web-based data formats is that they often represent information in different structures. In HTML files, tabular data is stored inside table elements, which require specialized packages

such as rvest to extract the data. JSON files, on the other hand, store information using nested objects and arrays, which must be parsed using packages such as jsonlite. Another challenge involves handling nested values such as lists of authors. In JSON format, multiple authors may be stored as an array, which must be converted into a format suitable for analysis within R.

Implementation of Data Import

The following code demonstrates how the HTML table and JSON file can be imported into R and converted into data frames for further analysis.

```{r}
library(rvest)
library(jsonlite)
library(dplyr)
library(gt)
library(stringr)
```

The HTML file contains a table that lists the books and their attributes. Using the **rvest** package, the table can be extracted and converted into a data frame.

```{r}
# HTML file
html_url <- "https://raw.githubusercontent.com/I
```

```
books_html <- read_html(html_url) |>
  html_table(fill = TRUE)

books_html_df <- books_html[[1]]

books_html_df |>
  head() |>
  gt()
```

Here is your **Quarto approach submission**, written to closely match the structure and tone of your provided template, but adapted to your NYT API assignment and topic.

Objective

The objective of this assignment is to use a New York Times API to retrieve JSON data, import it into R, and transform it into a tidy data frame for analysis. The goal is to explore patterns in article engagement by examining whether articles with images appear more frequently in the New York Times Most Popular list compared to those without images.

Approach

For this assignment, I used the **New York Times Most Popular API**, specifically the endpoint that returns the most viewed articles over the past 7 days.

The research question for this analysis is:

- **Do articles with images receive more engagement than those without in the New York Times Most Popular list?**

The dataset is obtained dynamically through an API request and includes information about the most viewed articles published by The New York Times.

The dataset contains the following variables:

- **Title** – Headline of the article
- **Section** – Category of the article (e.g., World, Technology, Politics)
- **Published Date** – Date the article was published
- **Byline** – Author(s) of the article
- **Media** – Nested field containing multimedia elements such as images

To answer the research question, the **media** field will be used to determine whether an article contains images. A new variable will be created:

- **Has Image** – A binary variable indicating whether the article includes at least one image (Yes/No)

The data retrieved from the API is in **JSON format**, which represents hierarchical and nested data commonly used in web services. This requires parsing and transformation before it can be analyzed in R.

Data Analysis Approach

The analysis will follow these steps:

1. **API Authentication**
   The API key will be securely accessed using an environment variable (`Sys.getenv`) to avoid hardcoding sensitive information.

2. **Data Retrieval**
   A request will be made to the NYT Most Popular API to retrieve the most viewed articles over a 7-day period.
3. **JSON Parsing**
   The JSON response will be parsed into an R object using the `jsonlite` package.
4. **Data Transformation**
   The relevant fields (title, section, published date, byline, and media) will be extracted and converted into a tidy data frame.
5. **Feature Engineering**
   A new variable (**Has Image**) will be created by checking whether the nested media field contains data.
6. **Data Cleaning**
   - Nested structures will be simplified
   - Missing media values will be treated as "No Image"
   - Dates will be converted into proper date format
7. **Exploratory Data Analysis**
   - Count and compare the number of articles with and without images
   - Analyze distribution across sections
   - Visualize differences using bar charts
8. **Interpretation**
   The results will be used to evaluate whether the presence of images is associated with higher representation in the Most Popular list, serving as a proxy for engagement.

Anticipated Challenges

One of the main challenges in this assignment is working with **nested JSON data** returned by the API. The media field, which contains image information, is stored as a list of nested

objects and requires additional processing to determine whether an article includes images.

Another challenge is that the API does not provide direct engagement metrics such as view counts. Instead, inclusion in the “Most Popular” list must be used as a proxy for engagement, which limits the ability to draw causal conclusions.

Additionally, handling missing or empty fields and converting them into meaningful variables (such as the binary image indicator) requires careful data cleaning and transformation.

Implementation of Data Import

The following code demonstrates how the JSON data is retrieved from the New York Times API and prepared for analysis in R.

‹/› {r}

```
library(tidyr)
library(ggplot2)
```

The API request retrieves the most viewed articles over the past 7 days. The API key is stored securely as an environment variable.

‹/› {r}

```
# Retrieve API key
api_key <- Sys.getenv("NYT_API_KEY")
```

```
# API request
url <- paste0(
  "https://api.nytimes.com/svc/mostpopular/v2/viewed/7.json?api-key=",
  api_key
)

response <- GET(url)

# Convert response to text
data_json <- content(response, as = "text", encoding = "UTF-8")

# Parse JSON
data <- fromJSON(data_json, flatten = TRUE)

# Extract results
articles <- data$results

# Convert to data frame
df <- articles %>%
  select(title, section, published_date, byline, media)

df |>
  head()
```

explain this line api_key <- Sys.getenv("NYT_API_KEY") and give me step by step instruction to store the key

🔍 Explanation of the Line

```r
api_key <- Sys.getenv("NYT_API_KEY")
```

This line retrieves your **New York Times API key** from your system's environment variables.

- `Sys.getenv("NYT_API_KEY")`
  → Looks for an environment variable named **"NYT_API_KEY"** on your system
- If it exists, it returns the stored value (your API key)
- That value is then assigned to the variable `api_key` in R

💡 Why this is important:

- Keeps your API key **secure**
- Prevents accidentally sharing it (e.g., in GitHub or assignments)
- Follows best practices required by your assignment

🔐 Step-by-Step: How to Store Your API Key

✅ Option 1 (Best for Quarto / RStudio): Using `.Renviron`

Step 1: Open your `.Renviron` file

In R or RStudio, run:

```r
usethis::edit_r_environ()
```

👉 If prompted to install `usethis`, run:

```r
install.packages("usethis")
```

Step 2: Add your API key

In the file that opens, add this line:

```r
NYT_API_KEY=your_actual_api_key_here
```

⚠️ Important:

- No quotes `" "`
- No spaces around `=`

Step 3: Save the file

- Save and close the `.Renviron` file

Step 4: Restart R

This is required so R reloads environment variables.

In RStudio:

- Click **Session → Restart R**

Step 5: Verify it works

```r
Sys.getenv("NYT_API_KEY")
```

If successful, it will return your API key.

✅ Option 2 (Quick but Temporary)

If you just want a quick test:

```r
Sys.setenv(NYT_API_KEY = "your_actual_api_key_here")
```

⚠️ Note:

- This only works for the current session
- You'll need to run it every time you restart R

🚀 Summary

- `Sys.getenv()` = **retrieve securely**
- `.Renviron` = **store permanently**
- `Sys.setenv()` = **temporary storage**

If you want, I can also:
✅ Show how to **hide API keys in Quarto when knitting**
✅ Help you **debug if your API key isn't working**

what do you mean by systems environment variables

Ah, perfect question! Let's break it down clearly.

🌐 What Are System Environment Variables?

System environment variables are **key-value pairs stored by your operating system** that programs (like R, Python, or your web browser) can access.

Think of them as **global settings** that any program on your computer can read to get information it needs, without hardcoding it inside your program.

🔑 Examples

- `PATH` → tells your computer where to find executable programs
- `HOME` → the path to your home directory
- `TEMP` → folder for temporary files
- `NYT_API_KEY` → (your own variable) stores your NYT API key

💡 Why Use Environment Variables for API Keys?

1. **Security** – Your API key is private. You don't put it directly in your code (like `api_key <- "abcd1234"`)
2. **Portability** – You can share your R scripts without exposing secret keys
3. **Convenience** – Programs can access the key automatically without rewriting code

---

🏗️ How It Works in R

1. You **store** your API key in a system environment variable called `NYT_API_KEY`
2. In R, you use:

```R
api_key <- Sys.getenv("NYT_API_KEY")
```

- `Sys.getenv("NYT_API_KEY")` looks up the value from your system
- Assigns it to `api_key` for use in your API calls

Now your R script can safely use the API key without ever exposing it directly in the code.

If you want, I can draw a **small diagram showing R, the system, and the environment variable** to make it super visual and easy to remember.

Do you want me to do that?

how to set NYT_API_KEY?

Here's a **step-by-step guide** to set your `NYT_API_KEY` so R can access it securely. There are two main ways: **permanent** (recommended) and **temporary** (for the current session only).

Option 1: Permanent – Using `.Renviron` (Best Practice)

This will keep your API key safe across all R projects.

Step 1: Open your `.Renviron` file

In R or RStudio, run:

```R
usethis::edit_r_environ()
```

⚠ If you don't have `usethis` installed, first run:

```R
```

```
install.packages("usethis")
```

This opens a text file where you can store environment variables.

Step 2: Add your NYT API key

At the end of the file, type:

```
NYT_API_KEY=your_actual_api_key_here
```

- Replace `your_actual_api_key_here` with the API key you got from the NYT developer portal.
- **Do not** put quotes (`" "`).
- **Do not** add spaces around the `=` sign.

Step 3: Save the file and restart R

- Save `.Renviron` and close it.
- Restart R or RStudio so it reloads the environment variables.

Step 4: Verify

‹/› R

```
Sys.getenv("NYT_API_KEY")
```

It should return your API key. If yes, you're done! ✅

Option 2: Temporary – For Current R Session Only

If you just want to test or don't want to save it permanently:

```r
Sys.setenv(NYT_API_KEY = "your_actual_api_key_here")
```

- Works only in the current R session
- Once you close R, you'll need to run it again

✅ Summary

Method	Duration	How to Use
`.Renviron`	Permanent	`Sys.getenv("NYT_API_KEY")`
`Sys.setenv()`	Current session	`Sys.getenv("NYT_API_KEY")`

If you want, I can now **show the full R code using** `NYT_API_KEY` to pull the Most Popular articles and check for images—ready to run after you set the key.

Do you want me to do that?